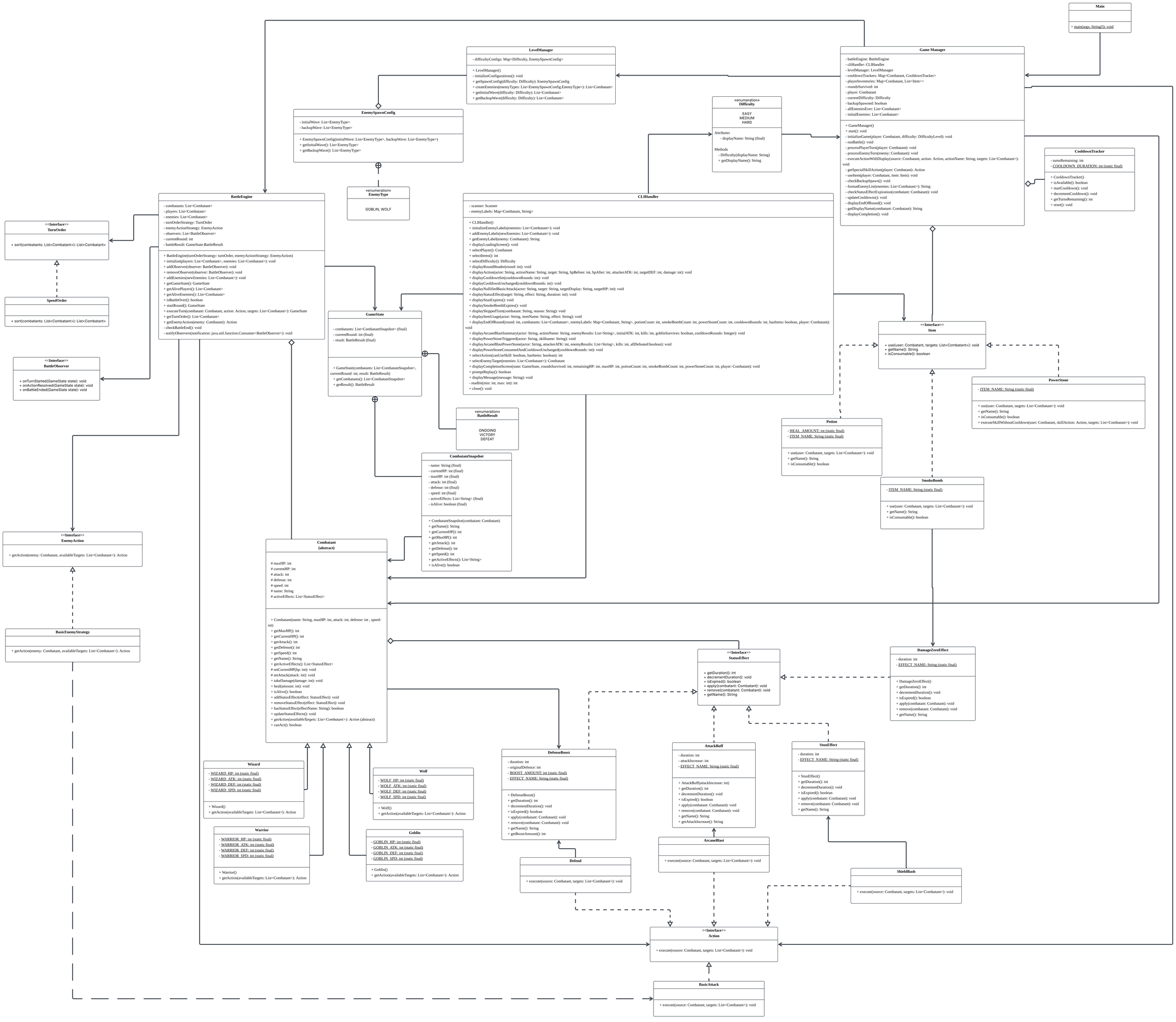
Main
+ main(args: String[]): void
LevelManager
- difficultyConfigs: Map<Difficulty, EnemySpawnConfig>
+ LevelManager()
- initializeConfigurations(): void
+ getSpawnConfig(difficulty: Difficulty): EnemySpawnConfig
+ createEnemies(enemyTypes: List<EnemySpawnConfig.EnemyType>): List<Combatant>
+ getInitialWave(difficulty: Difficulty): List<Combatant>
+ getBackupWave(difficulty: Difficulty): List<Combatant>
Game Manager
- battleEngine: BattleEngine
- cliHandler: CLIHandler
- levelManager: LevelManager
- cooldownTrackers: Map<Combatant, CooldownTracker>
- playerInventories: Map<Combatant, List<Item>>
- roundsSurvived: int
- player: Combatant
- currentDifficulty: Difficulty
- backupSpawned: boolean
- allEnemiesEver: List<Combatant>
- initialEnemies: List<Combatant>
+ GameManager()
+ start(): void
- initializeGame(player: Combatant, difficulty: DifficultyLevel): void
- runBattle(): void
- processPlayerTurn(player: Combatant): void
- processEnemyTurn(enemy: Combatant): void
- executeActionWithDisplay(source: Combatant, action: Action, actionName: String, targets: List<Combatant>): void
- getSpecialSkillAction(player: Combatant): Action
- useItem(player: Combatant, item: Item): void
- checkBackupSpawn(): void
- formatEnemyList(enemies: List<Combatant>): String
- checkStatusEffectExpiration(combatant: Combatant): void
- updateCooldowns(): void
- displayEndOfRound(): void
- getDisplayName(combatant: Combatant): String
- displayCompletion(): void
EnemySpawnConfig
- initialWave: List<EnemyType>
- backupWave: List<EnemyType>
+ EnemySpawnConfig(initialWave: List<EnemyType>, backupWave: List<EnemyType>)
+ getInitialWave(): List<EnemyType>
+ getBackupWave(): List<EnemyType>
«enumeration»
Difficulty
EASY
MEDIUM
HARD
Attributes
- displayName: String (final)
Methods
- Difficulty(displayName: String)
+ getDisplayName(): String
«enumeration»
EnemyType
GOBLIN, WOLF
CooldownTracker
- turnsRemaining: int
- COOLDOWN_DURATION: int (static final)
+ CooldownTracker()
+ isAvailable(): boolean
+ startCooldown(): void
+ decrementCooldown(): void
+ getTurnsRemaining(): int
+ reset(): void
BattleEngine
- combatants: List<Combatant>
- players: List<Combatant>
- enemies: List<Combatant>
- turnOrderStrategy: TurnOrder
- enemyActionStrategy: EnemyAction
- observers: List<BattleObserver>
- currentRound: int
- battleResult: GameState.BattleResult
+ BattleEngine(turnOrderStrategy: turnOrder, enemyActionStrategy: EnemyAction)
+ initialize(players: List<Combatant>, enemies: List<Combatant>): void
+ addObserver(observer: BattleObserver): void
+ removeObserver(observer: BattleObserver): void
+ addEnemies(newEnemies: List<Combatant>): void
+ getGameState(): GameState
+ getAlivePlayers(): List<Combatant>
+ getAliveEnemies(): List<Combatant>
+ isBattleOver(): boolean
+ startRound(): GameState
+ executeTurn(combatant: Combatant, action: Action, targets: List<Combatant>): GameState
+ getTurnOrder(): List<Combatant>
+ getEnemyAction(enemy: Combatant): Action
- checkBattleEnd(): void
- notifyObservers(notification: java.util.function.Consumer<BattleObserver>): void
CLIHandler
- scanner: Scanner
- enemyLabels: Map<Combatant, String>
+ CLIHandler()
+ initializeEnemyLabels(enemies: List<Combatant>): void
+ addEnemyLabels(newEnemies: List<Combatant>): void
+ getEnemyLabel(enemy: Combatant): String
+ displayLoadingScreen(): void
+ selectPlayer(): Combatant
+ selectItems(): int
+ selectDifficulty(): Difficulty
+ displayRoundHeader(round: int): void
+ displayAction(actor: String, actionName: String, target: String, hpBefore: int, hpAfter: int, attackerATK: int, targetDEF: int, damage: int): void
+ displayCooldownSet(cooldownRounds: int): void
+ displayCooldownUnchanged(cooldownRounds: int): void
+ displayNullifiedBasicAttack(actor: String, target: String, targetDisplay: String, targetHP: int): void
+ displayStatusEffect(target: String, effect: String, duration: int): void
+ displayStunExpires(): void
+ displaySmokeBombExpires(): void
+ displaySkippedTurn(combatant: String, reason: String): void
+ displayItemUsage(actor: String, itemName: String, effect: String): void
+ displayEndOfRound(round: int, combatants: List<Combatant>, enemyLabels: Map<Combatant, String>, potionCount: int, smokeBombCount: int, powerStoneCount: int, cooldownRounds: int, hasItems: boolean, player: Combatant): void
+ displayArcaneBlastSummary(actor: String, actionName: String, enemyResults: List<String>, initialATK: int, kills: int, goblinSurvives: boolean, cooldownRounds: Integer): void
+ displayPowerStoneTriggered(actor: String, skillname: String): void
+ displayArcaneBlastPowerStone(actor: String, attackerATK: int, enemyResults: List<String>, kills: int, allDefeated:boolean): void
+ displayPowerStoneConsumedAndCooldownUnchanged(cooldownRounds: int): void
+ selectAction(canUseSkill: boolean, hasItems: boolean): int
+ selectEnemyTarget(enemies: List<Combatant>): Combatant
+ displayCompletionScreen(state: GameState, roundsSurvived: int, remainingHP: int, maxHP: int, potionCount: int, smokeBombCount: int, powerStoneCount: int, player: Combatant): void
+ promptReplay(): boolean
+ displayMessage(message: String): void
- readInt(min: int, max: int): int
+ close(): void
<<Interface>>
TurnOrder
+ sort(combatants: List<Combatant>): List<Combatant>
SpeedOrder
+ sort(combatants: List<Combatant>): List<Combatant>
GameState
- combatants: List<CombatantSnapshot> (final)
- currentRound: int (final)
- result: BattleResult (final)
+ GameState(combatants: List<CombatantSnapshot>, currentRound: int, result: BattleResult)
+ getCombatants(): List<CombatantSnapshot>
+ getResult(): BattleResult
<<Interface>>
BattleObserver
+ onTurnStarted(GameState state): void
+ onActionResolved(GameState state): void
+ onBattleEnded(GameState state): void
«enumeration»
BattleResult
ONGOING
VICTORY
DEFEAT
CombatantSnapshot
- name: String (final)
- currentHP: int (final)
- maxHP: int (final)
- attack: int (final)
- defense: int (final)
- speed: int (final)
- activeEffects: List<String> (final)
- isAlive: boolean (final)
+ CombatantSnapshot(combatant: Combatant)
+ getName(): String
+ getCurrentHP(): int
+ getMaxHP(): int
+ getAttack(): int
+ getDefense(): int
+ getSpeed(): int
+ getActiveEffects(): List<String>
+ isAlive(): boolean
<<Interface>>
Item
+ use(user: Combatant, targets: List<Combatant>): void
+ getName(): String
+ isConsumable(): boolean
PowerStone
- ITEM_NAME: String (static final)
+ use(user: Combatant, targets: List<Combatant>): void
+ getName(): String
+ isConsumable(): boolean
+ executeSkillWithoutCooldown(user: Combatant, skillAction: Action, targets: List<Combatant>): void
Potion
- HEAL_AMOUNT: int (static final)
- ITEM_NAME: String (static final)
+ use(user: Combatant, targets: List<Combatant>): void
+ getName(): String
+ isConsumable(): boolean
SmokeBomb
- ITEM_NAME: String (static final)
+ use(user: Combatant, targets: List<Combatant>): void
+ getName(): String
+ isConsumable(): boolean
<<Interface>>
EnemyAction
+ getAction(enemy: Combatant, availableTargets: List<Combatant>): Action
Combatant
(abstract)
maxHP: int
currentHP: int
attack: int
defense: int
speed: int
name: String
activeEffects: List<StatusEffect>
+ Combatant(name: String, maxHP: int, attack: int, defense: int , speed: int)
+ getMaxHP(): int
+ getCurrentHP(): int
+ getAttack(): int
+ getDefense(): int
+ getSpeed(): int
+ getName(): String
+ getActiveEffects(): List<StatusEffect>
# setCurrentHP(hp: int): void
# setAttack(attack: int): void
+ takeDamage(damage: int): void
+ heal(amount: int): void
+ isAlive(): boolean
+ addStatusEffect(effect: StatusEffect): void
+ removeStatusEffect(effect: StatusEffect): void
+ hasStatusEffect(effectName: String): boolean
+ updateStatusEffects(): void
+ getAction(availableTargets: List<Combatant>): Action (abstract)
+ canAct(): boolean
BasicEnemyStrategy
+ getAction(enemy: Combatant, availableTargets: List<Combatant>): Action
<<Interface>>
StatusEffect
+ getDuration(): int
+ decrementDuration(): void
+ isExpired(): boolean
+ apply(combatant: Combatant): void
+ remove(combatant: Combatant): void
+ getName(): String
DamageZeroEffect
- duration: int
- EFFECT_NAME: String (static final)
+ DamageZeroEffect()
+ getDuration(): int
+ decrementDuration(): void
+ isExpired(): boolean
+ apply(combatant: Combatant): void
+ remove(combatant: Combatant): void
+ getName(): String
Wizard
- WIZARD_HP: int (static final)
- WIZARD_ATK: int (static final)
- WIZARD_DEF: int (static final)
- WIZARD_SPD: int (static final)
+ Wizard()
+ getAction(availableTargets: List<Combatant>): Action
Wolf
- WOLF_HP: int (static final)
- WOLF_ATK: int (static final)
- WOLF_DEF: int (static final)
- WOLF_SPD: int (static final)
+ Wolf()
+ getAction(availableTargets: List<Combatant>): Action
DefenseBoost
- duration: int
- originalDefence: int
- BOOST_AMOUNT: int (static final)
- EFFECT_NAME: String (static final)
+ DefenseBoost()
+ getDuration(): int
+ decrementDuration(): void
+ isExpired(): boolean
+ apply(combatant: Combatant): void
+ remove(combatant: Combatant): void
+ getName(): String
+ getBoostAmount(): int
AttackBuff
- duration: int
- attackIncrease: int
- EFFECT_NAME: String (static final)
+ AttackBuff(attackIncrease: int)
+ getDuration(): int
+ decrementDuration(): void
+ isExpired(): boolean
+ apply(combatant: Combatant): void
+ remove(combatant: Combatant): void
+ getName(): String
+ getAttackIncrease(): String
StunEffect
- duration: int
- EFFECT_NAME: String (static final)
+ StunEffect()
+ getDuration(): int
+ decrementDuration(): void
+ isExpired(): boolean
+ apply(combatant: Combatant): void
+ remove(combatant: Combatant): void
+ getName(): String
Warrior
- WARRIOR_HP: int (static final)
- WARRIOR_ATK: int (static final)
- WARRIOR_DEF: int (static final)
- WARRIOR_SPD: int (static final)
+ Warrior()
+ getAction(availableTargets: List<Combatant>): Action
Goblin
- GOBLIN_HP: int (static final)
- GOBLIN_ATK: int (static final)
- GOBLIN_DEF: int (static final)
- GOBLIN_SPD: int (static final)
+ Goblin()
+ getAction(availableTargets: List<Combatant>): Action
Defend
+ execute(source: Combatant, targets: List<Combatant>): void
ArcaneBlast
+ execute(source: Combatant, targets: List<Combatant>): void
ShieldBash
+ execute(source: Combatant, targets: List<Combatant>): void
<<Interface>>
Action
+ execute(source: Combatant, targets: List<Combatant>): void
BasicAttack
+ execute(source: Combatant, targets: List<Combatant>): void